```
.await(function(error, file1, file2) {
        createForceLayout(file1, file2);
});

function createForceLayout(nodes,edges) {
   var nodeHash = {};
   for (x in nodes) {
     nodeHash[nodes[x].id] = nodes[x];
   };
   for (x in edges) {
     edges[x].weight = parseInt(edges[x].weight);
     edges[x].source = nodeHash[edges[x].source];
     edges[x].target = nodeHash[edges[x].target];
   };

  var weightScale = d3.scale.linear()
     .domain(d3.extent(edges, function(d) {return d.weight;}))
     .range([.1,1]);

  var force = d3.layout.force().charge(-1000)
     .size([500,500])
     .nodes(nodes)
     .links(edges)
     .on("tick", forceTick);

d3.select("svg").selectAll("line.link")
   .data(edges, function (d) {return d.source.id + "-" + d.target.id;})
   .enter()
   .append("line")
   .attr("class", "link")
   .style("stroke", "black")
   .style("opacity", .5)
   .style("stroke-width", function(d) {return d.weight});

var nodeEnter = d3.select("svg").selectAll("g.node")
   .data(nodes, function (d) {return d.id})
   .enter()
   .append("g")
   .attr("class", "node");

nodeEnter.append("circle")
   .attr("r", 5)
   .style("fill", "lightgray")
   .style("stroke", "black")
   .style("stroke-width", "1px");

nodeEnter.append("text")
   .style("text-anchor", "middle")
   .attr("y", 15)
   .text(function(d) {return d.id;});

force.start();

function forceTick() {
  d3.selectAll("line.link")
     .attr("x1", function (d) {return d.source.x;})
     .attr("x2", function (d) {return d.target.x;})
     .attr("y1", function (d) {return d.source.y;})
     .attr("y2", function (d) {return d.target.y;});
```

**How much each node pushes away each other; if set to a positive value, nodes attract each other**

**"tick" events are fired continuously, running the associated function.**

**Key values for your nodes and edges will help when we update the network later.**

**Initializing the network starts firing "tick" events and calculates the degree centrality of nodes.**

**The tick function updates the edge-drawing code and node-drawing code based on the newly calculated node positions.**